Data Science HW2 — Discussion Questions

Regression, Binary Classification, Multiclass Classification

Aria Mokhtari Varnousfaderani 401102489

Contents

Part 1: Regression Methods Discussion

Dataset target: `daily_social_media_hours` (continuous, non-negative, contains zeros).

The regression results table (summarised below) is referenced throughout these answers.

Model	MSE	MAE	MAPE	R^2	Huber	RMSLE
Linear Regression	3.4930	1.5648	53.74	0.0457	1.1285	0.3903
Kernel Regression	3.9710	1.6594	52.30	-0.0849	1.2102	0.4014
Ridge Regression	3.4931	1.5649	53.74	0.0457	1.1285	0.3903
LASSO	3.4766	1.5608	53.59	0.0502	1.1273	0.3895
Polynomial Regression	3.7340	1.6186	53.23	-0.0201	1.1835	0.3949
Bayesian Ridge	3.5199	1.5694	53.93	0.0384	1.1358	0.3915
Elastic Net	3.4645	1.5587	53.50	0.0535	1.1246	0.3890
Decision Tree Reg.	3.6283	1.6052	52.97	0.0088	1.1733	0.3893
SVR	4.8976	1.8826	59.34	-0.3380	1.4235	0.4506
LWR	13.3866	2.9261	86.03	-2.6571	2.4690	0.7714

Table 1: Regression evaluation metrics (10% test set, `random_state=42`).

Q1.1. Best regression metric and justification — Answer: For this dataset I would choose **MAE (Mean Absolute Error)** as the primary evaluation metric, with **Huber loss** as a secondary check.

Why not MAPE? MAPE divides by the true value, so it explodes whenever `daily_social_media_hou` = 0 (some users have zero screen time). Our results confirm this: MAPE reaches 86% for LWR, which is essentially meaningless. I computed MAPE only on the non-zero subset (the `mask = yt != 0` trick in the code), but even then the values are 52–54%, which shows the metric is unreliable here.

Why not MSE/RMSE? MSE squares errors, so a few large outliers (people with extreme social media habits) dominate the score and make small differences between models look big.

Why MAE? MAE gives the average absolute error in the same units as the target (hours per day), which is easy to interpret. It is also more robust to outliers than MSE. Looking at the results, *Elastic Net* gives the lowest MAE (1.5587), closely followed by LASSO (1.5608) — a difference of only 0.002, which suggests the linear + regularisation family is consistently the best group.

Why also Huber? Huber loss behaves like MSE for small errors and like MAE for large errors (using $\delta = 1.0$ in the code). It confirms the ranking: Elastic Net best (1.1246), LWR worst (2.4690).

Overall, **MAE is the most honest and interpretable metric** for a non-negative, zero-containing, social-media hours target.

Q1.2. When each regression model is preferable — Answer:

Linear Regression
Best when the relationship between features and target is approximately linear and there is no serious multicollinearity. It is the baseline: fast, interpretable, and assumption-transparent.

Ridge Regression
Preferred when there are many correlated features (multicollinearity). It adds an L2 penalty that shrinks all coefficients smoothly but keeps them all in the model. In our results Ridge and Linear are almost identical ($R^2 = 0.0457$), suggesting multicollinearity is not a big problem in this dataset.

LASSO Preferred when you suspect only a few features truly matter and you want automatic feature selection. The L1 penalty drives irrelevant coefficients exactly to zero. LASSO slightly outperforms Ridge here ($R^2 = 0.0502$ vs 0.0457), hinting that some features are indeed irrelevant.

Elastic Net
Combines L1 and L2 penalties, so it handles correlated features better than LASSO while still producing sparsity. It gives the best R^2 (0.0535) and lowest MAE in our experiment, making it the recommended model overall.

Polynomial Regression
Useful when the true relationship is non-linear but still smooth and low-degree. Here it performed worse than linear models ($R^2 = -0.0201$), indicating overfitting to noise.

Bayesian Ridge
Preferred when you need uncertainty estimates on predictions or you have a small dataset and want principled regularisation through priors.

Kernel / SVR
Best when there are strong non-linear patterns that cannot be captured by polynomial expansion. In our data SVR performed poorly ($R^2 = -0.338$), which suggests the kernel did not match the underlying structure or hyperparameters needed more tuning.

Decision Tree Regression
Good when the relationship is piecewise constant or there are sharp decision boundaries. It is interpretable but prone to overfitting. Our result ($R^2 = 0.0088$) is near zero, indicating poor generalisation.

Locally Weighted Regression (LWR)
Useful when the relationship changes a lot across different regions of the feature space (non-stationarity). However, it is very sensitive to the bandwidth parameter and does not scale well. Its catastrophic result here ($R^2 = -2.66$, MAE $= 2.93$) likely reflects a badly tuned bandwidth.

Q1.3. The kernel trick and how it helps regression — Answer: In its standard form, a model like SVR tries to find a hyperplane that fits the data in the original feature space. The **kernel trick** implicitly maps the input features $\mathbf{x}$ into a much

higher-dimensional (possibly infinite-dimensional) space $\phi(\mathbf{x})$ *without ever computing* ϕ *explicitly*. Instead, every computation only needs the inner product $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$, which is replaced by a cheap *kernel function* $K(\mathbf{x}_i, \mathbf{x}_j)$, such as the RBF kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$.

This helps regression because real-world data is often **non-linearly** related to the target. By working in the transformed space the model can fit curves, spirals, and other complex shapes that a straight line cannot capture, while still solving a *linear* optimisation problem in the transformed space — giving the best of both worlds: expressiveness and tractability.

In our experiment the kernel SVR underperformed, which tells us that either the RBF kernel was not the right choice for social-media usage data, or the kernel hyperparameters (bandwidth γ, regularisation C) needed cross-validated tuning.

Part 2: Binary Classification Methods Discussion

Dataset target: `depression_label` (binary, clearly imbalanced — see below).

Model	Acc.	Prec.	Rec.	F1	AUC	Sens.	Spec.
Logistic Regression	0.975	1.000	0.40	0.571	0.993	0.40	1.000
Linear SVM	0.975	1.000	0.40	0.571	0.991	0.40	1.000
Kernel SVM	0.958	0.500	0.20	0.286	0.974	0.20	0.991
KNN	0.958	0.500	0.20	0.286	0.596	0.20	0.991
Decision Tree	0.992	0.833	1.00	0.909	0.996	1.00	0.991
Random Forest	0.958	0.000	0.00	0.000	0.998	0.00	1.000
LDA	0.958	0.000	0.00	0.000	0.983	0.00	1.000
Naive Bayes	0.983	1.000	0.60	0.750	0.990	0.60	1.000

Table 2: Binary classification results (`depression_label`, 10% test set).

Q2.1. Best classification metric and justification — Answer: I would choose **AUC-ROC** as the primary metric and **F1-Score** as the secondary metric.

Why not Accuracy? The dataset is clearly **imbalanced**. Models like Random Forest and LDA predict *every single sample as the majority class*, getting $F1 = 0$, yet they still report Accuracy = 0.958. That is exactly the kind of misleading result accuracy produces in imbalanced settings: a completely useless model looks like a 95.8% success.

Why AUC-ROC? AUC measures the model's ability to *rank* positives above negatives across all possible decision thresholds. It is threshold-independent and completely unaffected by class imbalance in the sense that a random classifier always gets 0.5 regardless of the ratio. The results show Random Forest (AUC = 0.998) actually has excellent discriminative power — it just needs a different threshold. AUC reveals this, while Accuracy hides it.

Why also F1? F1 combines Precision and Recall into one number at the default threshold. It penalises models that ignore the minority class. The Decision Tree gives the best F1 (0.909) with perfect Recall (1.0) and good Precision (0.833), making it the practically best-performing model.

In a depression-detection context (medical/health data), **missing a true positive is dangerous**, so high Recall / Sensitivity matters more than high Precision. This further supports preferring F1 and AUC over Accuracy.

Q2.2. Three techniques to regularise the training of Decision Trees — Answer:

1. **Maximum depth** (`max_depth`) — Limits how deep the tree can grow. A shallow tree cannot memorise individual training points, so it is forced to learn more general patterns. However, depth alone is not always enough: even a shallow tree can overfit if the data contains many uninformative splits near the root.
2. **Minimum samples per leaf** (`min_samples_leaf`) — Prevents the tree from creating a leaf node unless it contains at least k samples. This stops the tree from fitting tiny,

noisy subgroups. It is often more effective than `max_depth` because it directly controls how many data points support each decision.

3. **Cost-complexity pruning (post-pruning, α)** — After growing a full tree, branches are pruned back if they do not sufficiently reduce the impurity per added node (controlled by `ccp_alpha` in scikit-learn). Unlike the previous two which prevent growth (pre-pruning), this technique grows first and simplifies afterwards, and often produces better-generalising trees because it can evaluate the combined effect of many splits.

Q2.3. Linear SVM vs. Kernel SVM — Answer:

Decision Boundary

Linear SVM finds a *hyperplane* (a straight line in 2D) that maximises the margin between classes. *Kernel SVM* uses the kernel trick to find a *non-linear* boundary in the original space by working linearly in a transformed feature space.

Computational Cost

Linear SVM scales well to large datasets (even millions of samples with solvers like `LinearSVC`). Kernel SVM has training complexity roughly $O(n^2)$–$O(n^3)$, so it becomes slow on large datasets.

Interpretability

Linear SVM has directly interpretable feature weights. Kernel SVM weights live in the kernel space and are not directly interpretable in terms of original features.

When to prefer Linear SVM

When features are already linearly separable (or close to it), when the dataset is large, or when interpretability is required. Our results show Linear SVM and Logistic Regression tied (Acc = 0.975, $F1 = 0.571$, AUC = 0.991), which suggests the `depression_label` classes are nearly linearly separable.

When to prefer Kernel SVM

When the classes are not linearly separable and a non-linear boundary is needed. In our experiment, Kernel SVM actually performed *worse* ($F1 = 0.286$, AUC = 0.974), suggesting the RBF kernel may have overfit or its hyperparameters needed tuning.

Part 3: Multiclass Classification Methods Discussion

Dataset target: `social_interaction_level` (3 classes, roughly balanced).

Model	Acc.	F1 Mac.	F1 Mic.	F1 Wt.	Kappa	LogLoss	Top-2
SVM (OVO)	0.333	0.325	0.333	0.323	-0.001	1.098	0.717
LR OVR	0.392	0.378	0.392	0.387	0.076	1.098	0.708
LR Multinomial	0.383	0.370	0.383	0.378	0.064	1.097	0.708
KNN	0.342	0.333	0.342	0.339	0.009	1.155	0.675
Decision Tree	0.300	0.298	0.300	0.299	-0.052	23.82	0.667
XGBoost	0.333	0.317	0.333	0.326	-0.010	1.422	0.642
LightGBM	0.383	0.366	0.383	0.376	0.065	1.741	0.667
AdaBoost	0.375	0.366	0.375	0.370	0.054	1.099	0.717
Random Forest	0.350	0.333	0.350	0.343	0.012	1.123	0.692
Voting Classifier	0.367	0.352	0.367	0.361	0.038	1.114	0.675

Table 3: Multiclass classification results (10% test set, 3-class target).

Note: All models perform near random chance (≈ 33% for 3 equal classes), and Cohen's Kappa values are near zero. The target variable may not be predictable from the available features, or feature engineering is needed.

Q3.1. Best multiclass metric and justification — Answer: I would choose **Macro F1-Score** as the primary metric, supported by **Cohen's Kappa**.

Why Macro F1? Macro F1 computes F1 independently for each class and then averages them with *equal weight* regardless of class size. This is important for a 3-class social interaction level target where we want each class to be predicted well — not just the most common one. A model that ignores Class 0 completely can still get a high Weighted F1 if Class 0 is rare, but Macro F1 will immediately reveal the problem.

Looking at our results, Macro F1 and Accuracy tell very similar stories because the classes appear roughly balanced (the per-class precision and recall values are not dramatically different). LR OVR has the best Macro F1 (0.378) and the best Accuracy (0.392).

Why also Kappa? Cohen's Kappa measures agreement *beyond chance*. Since all models are close to 33% accuracy (the random-chance baseline for 3 classes), Kappa is useful to confirm whether any model is genuinely learning anything. Most Kappa values here are close to zero, which is a clear warning that the task is very hard with the current features.

Why not Micro F1? Micro F1 aggregates all true positives, false positives, and false negatives across classes before computing F1. For balanced classes, Micro F1 equals Accuracy, so it adds no new information.

Why not Log Loss? Log Loss is useful when calibrated probabilities matter (e.g., risk scoring). Here the Decision Tree gets an outlier Log Loss of 23.82 (probably due to zero-probability predictions), which would distort any ranking based on Log Loss alone.

Q3.2. Extending KNN and Decision Trees to multi-label classification — Answer: In a **multi-label** problem each sample can belong to *multiple* classes simultaneously

(e.g., a player can have both a heart condition and a knee injury at the same time). This is fundamentally different from multiclass, where each sample belongs to exactly one class.

KNN for multi-label: Standard KNN looks at the k nearest neighbours and returns the majority class. For multi-label, instead of a majority vote for a single label, we *aggregate all labels* present among the k neighbours. Each label ℓ is predicted as positive if the fraction of neighbours that carry label ℓ exceeds some threshold τ (often $\tau = 0.5$). This is called **ML-kNN** and is a natural extension because distance-based search naturally retrieves the most similar examples regardless of how many labels they carry.

Decision Trees for multi-label: A standard Decision Tree minimises impurity (e.g., Gini) for a single label. For multi-label, the impurity function is extended to measure impurity *across all labels simultaneously* (e.g., the sum of label-wise entropies, or the Hamming impurity). Each leaf then stores a *vector* of class probabilities — one probability per label — instead of a single class distribution. At prediction time, each label probability is compared to a threshold to decide which labels are "on".

Both extensions are fundamentally straightforward because neither algorithm requires the output space to be a single scalar; they just need a modified impurity/aggregation rule.

Q3.3. Multi-label football scenario: best accuracy metric and why — Answer: In the football problem each player can simultaneously belong to up to 4 binary labels:

C1: Has played for the national team.

C2: Has a history of heart problems.

C3: Has had previous knee injuries.

C4: Has been a team captain.

The four labels have very **different base rates**: Class 2 (heart problems) is almost certainly rare compared to Class 4 (captains). Because of this imbalance and because we have a multi-label setting, I would use **Hamming Loss** as the main metric and **Macro-averaged F1** across all four labels as a supporting metric.

Why Hamming Loss? Hamming Loss counts the fraction of individual (sample, label) pairs that are wrong (either a false positive or a false negative), divided by $n_{\text{samples}} \times n_{\text{labels}}$. It is the most natural multi-label accuracy metric because it treats each label-prediction independently. A model that gets 3 out of 4 labels right for every player has a Hamming Loss of 0.25, which is easy to interpret.

Why also Macro F1? Subset Accuracy (exact match — all 4 labels correct for a player) is too strict and can be zero even for a nearly perfect model. Macro F1 averages the F1 of each label independently, giving equal weight to rare labels like Class 2 (heart problems), which is medically the most important to detect correctly. A high Macro F1 ensures we are not ignoring the rare but critical labels.

Why not Subset / Exact-Match Accuracy? It requires every single label to be correct for a sample to count as correct. With 4 binary labels, this is extremely strict and produces artificially low scores even when the model is mostly correct.

Why not standard Accuracy? Standard accuracy is not even well-defined for multi-label problems without choosing a specific aggregation strategy, and it typically collapses to a per-label accuracy that hides the difficulty of predicting rare conditions.

Part 4: Challenging Questions (Bonus)

Q4.1. Bias–variance trade-off in regression — Answer: The expected prediction error of any regression model can be decomposed as:

$$\mathbb{E}\Big[(y - \hat{f}(x))^2\Big] = \underbrace{\text{Bias}^2[\hat{f}(x)]}_{\text{systematic error}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}} .$$

Bias is the error from wrong assumptions in the model. A linear model trying to fit a quadratic relationship has high bias: it is always wrong in a predictable way.

Variance is the error from sensitivity to small fluctuations in the training set. A high-degree polynomial that changes dramatically with each new training sample has high variance (overfitting).

The **trade-off** is that reducing one often increases the other. Simple models (Linear Regression) have low variance but high bias. Complex models (Decision Tree Regression at full depth) have low bias but high variance. Regularisation methods like Ridge, LASSO, and Elastic Net sit in the middle: they introduce a small amount of bias (by penalising large weights) in exchange for a large reduction in variance.

In our results, Elastic Net and LASSO achieve the best balance ($R^2 \approx 0.05$) while Polynomial Regression overfits ($R^2 = -0.02$) and LWR has catastrophic variance ($R^2 = -2.66$).

Q4.2. When does Kernel Regression outperform Linear Regression? — Answer: Kernel Regression outperforms Linear Regression when the true relationship between features and target is genuinely **non-linear** and when there is **enough data** to reliably estimate the kernel function. Specifically:

- When the scatter plot shows clear curves or clusters that a straight line cannot fit.
- When residual plots from Linear Regression show systematic patterns (not random scatter).
- When the dataset is large enough that the extra model complexity does not lead to overfitting.

In our experiment, Kernel Regression (`MSE` = 3.97) performed *worse* than Linear Regression (`MSE` = 3.49). This suggests either: (a) the relationship between the social media features and daily hours really is approximately linear; or (b) the kernel bandwidth parameter was poorly chosen. In practice, kernel-based methods need careful cross-validated hyperparameter search to outperform a well-regularised linear model.

Q4.3. L1 vs. L2 regularisation: LASSO vs. Ridge — Answer: Both add a penalty term to the loss function to prevent overfitting, but the geometry of the penalty is different.

$$\text{Ridge:} \quad \mathcal{L} + \lambda \sum_j w_j^2 \qquad \text{LASSO:} \quad \mathcal{L} + \lambda \sum_j |w_j|$$

When LASSO performs better: When the true model is *sparse*, i.e., only a few features are actually relevant. LASSO drives irrelevant weights exactly to zero, effectively selecting features automatically.

When Ridge performs better: When many features contribute roughly equally (no single dominant feature), or when features are highly correlated. Ridge distributes the penalty smoothly and never sets weights to exactly zero.

Why LASSO produces sparsity: Geometrically, the L1 constraint set is a diamond (a hypercube rotated 45°) with sharp corners at the coordinate axes. When the loss function's contours touch the constraint, they almost always hit a corner where many coefficients are exactly zero. The L2 constraint set is a smooth sphere with no corners, so the optimum can land anywhere on its surface — coefficients are shrunk but rarely zero.

In our results, LASSO ($R^2 = 0.0502$) slightly outperforms Ridge ($R^2 = 0.0457$), consistent with the idea that some features in this social media dataset are irrelevant and LASSO is pruning them.

Q4.4. Why MAPE is unreliable in some datasets — Answer: MAPE is defined as:

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

This formula breaks down in three situations:

1. **Zero true values:** Division by $y_i = 0$ produces infinity or undefined values. Our target `daily_social_media_hours` contains zeros (people who use no social media), so MAPE is undefined for those samples. We handle this with the mask trick in the code, but this biases the metric.
2. **Small true values:** Even when $y_i \neq 0$ but is very small (e.g., $y_i = 0.1$), an absolute error of 0.1 gives 100% MAPE, while the same error on a large value ($y_i = 5$) gives only 2% MAPE. This makes MAPE asymmetric and dominated by small-valued samples.
3. **Asymmetry:** Over-predicting by 50% gives a MAPE of 50%, but under-predicting by 50% also gives 50%. However, an over-prediction can theoretically be infinite ($+\infty$%), while an under-prediction is capped at 100% (you can never predict less than zero for a non-negative target).

In our results, MAPE ranges from 52% to 86% across models — values so high that they are practically uninterpretable. This confirms that MAPE is a poor choice for this dataset.

Q4.5. Effect of outliers on regression models — Answer: Outliers are data points that deviate far from the typical pattern. Their effect depends on which model and metric we use.

Linear/Ridge/LASSO: These minimise MSE or an MSE-like objective. Because MSE *squares* errors, a single outlier with a large residual e contributes e^2, which can dominate the total loss and pull the regression line strongly towards it. Ridge

and LASSO regularise the *weights*, not the residuals, so they offer no direct protection against outliers.

SVR: Uses an ϵ-insensitive tube. Points inside the tube contribute zero to the loss, and outliers beyond the tube are penalised linearly (not quadratically). This gives SVR some inherent robustness.

Decision Tree Regression: Outliers can cause splits to be placed in unusual positions, creating leaves with just one or two extreme samples. This inflates variance.

Huber Loss: A model trained with Huber loss treats small residuals quadratically and large residuals linearly, giving a good compromise between sensitivity to outliers (MSE) and robustness (MAE).

Metric choice: MSE amplifies outlier errors; MAE is far more robust. Our Huber and MAE rankings both agree: Elastic Net > LASSO, while LWR is catastrophically bad — likely because a few extreme social-media users dragged the locally weighted fits off course.

Q4.6. Class imbalance: why accuracy is misleading — Answer: Class imbalance means one class (the majority) contains far more samples than another (the minority). In our binary dataset, the positive `depression_label` class is rare (perhaps 5 out of 120 test samples).

Why accuracy is misleading: A model that predicts "no depression" for every single sample achieves $\approx 95.8\%$ accuracy (the majority class baseline), yet it detects *zero* true positives. Random Forest and LDA demonstrate this perfectly: Accuracy = 0.958 but $F1 = 0.000$ because they never predict the minority class.

Effect on binary metrics:

- **Precision** can be inflated if the model only predicts positive when it is very confident, avoiding false positives.
- **Recall / Sensitivity** collapses to zero if the model never predicts the positive class.
- **F1** is the harmonic mean of Precision and Recall, so it punishes zero recall severely.
- **AUC-ROC** is largely threshold-independent and measures the ranking quality across the full range of thresholds. Random Forest achieves AUC = 0.998, showing it has learned useful structure — it just needs a different decision threshold (or a cost-sensitive approach) to translate that into useful predictions.

Remedies: class weighting in the loss function, oversampling (SMOTE), undersampling, or adjusting the classification threshold.

Q4.7. Decision boundaries of the models (bonus) — Answer:

Logistic / Linear SVM / LDA
Linear hyperplane in feature space. One straight boundary separates all classes. The boundary is a line in 2D, a plane in 3D, etc.

Kernel SVM
: Non-linear curved boundary in the original space (e.g., a circle with RBF kernel). Linear in the kernel-transformed space.

KNN
: Irregular, non-parametric boundary that follows the local density of training points. Becomes more complex as k decreases.

Decision Tree
: Axis-aligned rectangular regions. All splits are parallel to feature axes, creating staircase-like boundaries. Cannot make diagonal splits without many levels.

Random Forest / XGBoost / LightGBM / AdaBoost
: Also axis-aligned (ensemble of trees), but by combining many trees the effective boundary becomes smoother and can approximate diagonal and curved boundaries.

Naive Bayes
: Elliptical (Gaussian NB) or step-function (Bernoulli NB) boundaries based on per-class feature distributions.

LR Multinomial vs. OVR
: Both are linear boundaries, but Multinomial learns all class boundaries jointly, while OVR learns K independent one-vs-rest boundaries. In practice, they are very similar for well-separated classes.

Q4.8. Effect of K in KNN — Answer: K controls how many neighbours vote on the prediction.

- **Small K (e.g., $K = 1$):** The model is very sensitive to individual training points. The decision boundary is highly irregular. It fits training data perfectly (zero training error for $K = 1$) but overfits: high variance, low bias.
- **Large K (e.g., $K = n$):** The model predicts the global majority class for every point. The boundary becomes very smooth but cannot capture local patterns: low variance, high bias.
- **Optimal K:** Lies somewhere in between and is found by cross-validation. A common heuristic is $K = \sqrt{n}$, but this is only a starting point.

In our binary results, KNN with a moderate default K gives $F1 = 0.286$ and AUC $= 0.596$, which is barely above random. This low AUC suggests that simple distance-based voting is not able to capture the structure in this dataset — possibly because the feature scales are very different (even after StandardScaler) or because the relevant patterns are non-local.

Q4.9. Overfitting in Decision Trees: depth, pruning — Answer: Why Decision Trees overfit easily: A decision tree can keep splitting until every leaf contains exactly one training sample (purity $= 1$). At that point, it has memorised the training set completely. Unlike parametric models, trees have no built-in regularisation; they grow greedily and will always prefer a split that reduces impurity, even if it only captures noise.

Why max_depth alone is not enough: Even a tree of depth 3 can overfit if the root splits on a noisy feature. Max depth does not prevent bad splits near the root — it only limits the total number of splits. A very wide but shallow tree can still have many thousands of leaves if features have many possible split points.

How pruning works:

- **Pre-pruning** (early stopping): Stop splitting when some criterion is met — e.g., the node has fewer than `min_samples_leaf` samples, or the impurity gain is below a threshold. Simple and fast but can stop too early.
- **Post-pruning (cost-complexity / α-pruning)**: Grow the full tree, then iteratively remove the branch that produces the smallest increase in impurity-per-removed-node, weighted by α. Cross-validate to find the best α. This is generally more powerful because it evaluates the full joint effect of a branch.

In our results, the Decision Tree is the *best* binary model ($F1 = 0.909$). This may partly be because the default scikit-learn stopping criteria already impose mild regularisation, but it also reflects that the tree found genuinely useful splits for the imbalanced `depression_label` target.

Q4.10. Why tree-based models are good feature selectors — Answer: At each split, a decision tree searches over all features and all possible split thresholds to find the one that reduces impurity (Gini or entropy) the most. Features that consistently appear near the top of the tree (close to the root) and reduce impurity by large amounts are, by definition, the most informative features.

Feature importance in tree-based models is computed by summing the weighted impurity reduction for each feature across all nodes where it was used:

$$\text{Importance}(f) = \sum_{\text{nodes using } f} \frac{n_{\text{node}}}{n_{\text{total}}} \cdot \big[\text{impurity before} - \text{weighted impurity after}\big].$$

In ensembles like Random Forest and XGBoost, this is averaged across hundreds of trees, which makes the importance estimates more stable and less sensitive to the specific random split of the training data.

Key advantage over filter methods (e.g., correlation): Tree importances capture non-linear relationships and interactions between features, not just pairwise linear correlation.

Limitation: Trees are biased towards features with many unique values (continuous or high-cardinality features appear important even when they are not). Permutation importance is a better alternative when unbiased selection is needed.

Q4.11. Micro vs. Macro vs. Weighted F1 — Answer: All three are ways to average per-class F1 scores in a multiclass setting.

Macro F1 Simple average of per-class F1 scores, giving *equal weight* to every class regardless of how many samples it has. **When Macro F1 is a better reflection:** When all classes are equally important and you want to know

if the model is good across the board. For our 3-class social interaction target (roughly balanced), Macro F1 is the most informative.

Micro F1 Aggregates all TP, FP, FN globally and then computes one F1. For balanced classes it equals Accuracy. **Why Micro F1 favours large classes:** Large classes contribute more TPs and TNs to the global counts, so a model that is great at the majority class and terrible at the minority class still gets a high Micro F1.

Weighted F1
Average of per-class F1 scores weighted by *support* (number of true instances of each class). Gives more weight to frequent classes. **When Weighted F1 is misleading:** In imbalanced datasets: a model that ignores rare classes can still achieve high Weighted F1 because those classes have little weight. It is not a reliable signal of model fairness across classes.

In our multiclass table, Macro and Weighted F1 are very close (e.g., LR OVR: Macro = 0.378, Weighted = 0.387), consistent with roughly balanced classes.

Q4.12. Multi-label vs. Multiclass: fundamental differences — Answer:

Aspect	Multiclass	Multi-label
Output space	Each sample has exactly one class label. Output is a single integer $y \in \{0, 1, \ldots, K-1\}$.	Each sample can have zero or more labels. Output is a binary vector $\mathbf{y} \in \{0,1\}^K$.
Loss function	Cross-entropy (Softmax output sums to 1).	Binary cross-entropy applied independently to each label (outputs are independent probabilities).
Thresholding	arg max over the K output logits gives the single predicted class.	Each label has its own threshold (often 0.5); all labels above threshold are predicted positive.
Metrics	Accuracy, Macro/Micro/Weighted F1, Kappa.	Hamming Loss, Subset Accuracy, Macro F1 per label, Label Ranking Average Precision.

Why KNN and Decision Trees can be extended: Both algorithms are *instance-based* or *split-based* and do not assume a single output scalar. KNN can aggregate a label vector from neighbours (ML-kNN); Decision Trees can store a label vector at each leaf and use a multi-label impurity measure. Neither requires a softmax normalisation across labels, making the multi-label extension natural.

Q4.13. Precision–recall trade-off — Answer: For any binary classifier with a continuous score (probability or decision function), the classification threshold τ controls the trade-off:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

- **Lowering** $\tau \Rightarrow$ more samples predicted positive $\Rightarrow$ Recall increases (fewer missed positives, more false negatives become true positives), but Precision decreases (more false positives).
- **Raising** $\tau \Rightarrow$ fewer samples predicted positive $\Rightarrow$ Precision increases (only very confident positives are predicted), but Recall decreases.

In our binary dataset (imbalanced, depression detection), the default threshold (0.5) is clearly too high. Logistic Regression gets Precision = 1.0 but Recall = 0.4. If we lowered the threshold, Recall would improve at the cost of some false alarms. For a medical application, **higher Recall** is usually preferred because missing a depressed patient is more dangerous than a false alarm.

Q4.14. ROC curve vs. PR curve — Answer:

ROC curve
Plots True Positive Rate (Recall / Sensitivity) on the y-axis against False Positive Rate (1 − Specificity) on the x-axis as the threshold varies. Area under the ROC curve (AUC) measures overall ranking quality. A random classifier gets AUC = 0.5.

PR curve Plots Precision on the y-axis against Recall on the x-axis. Area under the PR curve (AUPRC) measures performance specifically on the positive class.

When PR is more informative:
In **imbalanced** datasets. ROC is optimistic about imbalanced data because False Positive Rate is computed relative to the large number of true negatives, making it artificially small. A model can have a high AUC but a very low AUPRC when positives are rare. Our Random Forest (AUC = 0.998) looks perfect on ROC, but its $F1 = 0.000$ tells a very different story — the PR curve would immediately reveal the failure.

Rule of thumb:
For balanced datasets, use ROC. For imbalanced datasets (like ours), always inspect the PR curve alongside ROC.

Q4.15. If given unlimited time and resources, how would you improve? — Answer: Given unlimited time and resources, I would improve along four axes:

1. Better preprocessing:

- Perform deeper outlier analysis and apply robust scaling or Winsorisation instead of simple standardisation.
- Handle class imbalance in binary classification with SMOTE, ADASYN, or class-weighted loss functions — models like Random Forest are currently collapsing to the majority class.
- Investigate and impute missing values with model-based imputation (MICE / IterativeImputer) rather than mean/median.

2. Better features:

- Apply feature selection (e.g., recursive feature elimination, tree-based importance, or mutual information) to remove irrelevant or redundant columns — the near-zero R^2 in regression and near-random accuracy in multiclass suggest many features are noise.
- Engineer interaction features and domain-specific transformations (e.g., ratios, log-transforms for skewed columns).
- Use automated feature engineering tools (Featuretools).

3. Better models:

- Perform exhaustive cross-validated hyperparameter search (`Optuna` / `RandomizedSearchCV`) for all models, especially SVR (kernel, C, γ) and KNN (K, distance metric).
- Try neural networks (MLP, TabNet) for the multiclass task where all tree and linear models are stuck near random chance.
- For multi-label football classification, try Label Powerset or Classifier Chains to capture label correlations.

4. Better evaluation:

- Use stratified k-fold cross-validation instead of a single 90/10 split to get more reliable metric estimates.
- Report confidence intervals on all metrics.
- For the imbalanced binary task, evaluate AUPRC (PR curve area) alongside AUC-ROC.
- For regression, investigate residual plots and check whether heteroscedasticity or non-linearity is present before concluding that linear models are sufficient.